

**T** **B** *I* <> 🔗 🖼️ 💬 ⋮ ⋮ — Ψ 😊 ☰ Close

```
# Breast Cancer Classification using Machi

## Objective
The objective of this project is to build
classifies breast tumors as malignant or b
Wisconsin dataset.
```

# Breast Cancer Classification using Machine Learning

## Objective

The objective of this project is to build a machine learning model that classifies breast tumors as malignant or benign using the Breast Cancer Wisconsin dataset.

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
```

```
# Load the Breast Cancer Dataset
breast_cancer = load_breast_cancer()

# Create a DataFrame
df = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)

# Add the target column
df['target'] = breast_cancer.target

# Display the first 5 rows
df.head()
```

|   | mean<br>radius | mean<br>texture | mean<br>perimeter | mean<br>area | mean<br>smoothness | mean<br>compactness | mean<br>concavity | me<br>conca<br>poin |
|---|----------------|-----------------|-------------------|--------------|--------------------|---------------------|-------------------|---------------------|
| 0 | 17.99          | 10.38           | 122.80            | 1001.0       | 0.11840            | 0.27760             | 0.3001            | 0.147               |
| 1 | 20.57          | 17.77           | 132.90            | 1326.0       | 0.08474            | 0.07864             | 0.0869            | 0.070               |
| 2 | 19.69          | 21.25           | 130.00            | 1203.0       | 0.10960            | 0.15990             | 0.1974            | 0.127               |
| 3 | 11.42          | 20.38           | 77.58             | 386.1        | 0.14250            | 0.28390             | 0.2414            | 0.105               |
| 4 | 20.29          | 14.34           | 135.10            | 1297.0       | 0.10030            | 0.13280             | 0.1980            | 0.104               |

5 rows × 31 columns

```
# Display dataset information
print("Dataset Shape:", df.shape)

print("\nColumn Names:")
print(df.columns)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nTarget Names:")
print(breast_cancer.target_names)
```

Dataset Shape: (569, 31)

Column Names:

```
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
      'mean smoothness', 'mean compactness', 'mean concavity',
      'mean concave points', 'mean symmetry', 'mean fractal dimension',
      'radius error', 'texture error', 'perimeter error', 'area error',
      'smoothness error', 'compactness error', 'concavity error',
      'concave points error', 'symmetry error', 'fractal dimension error',
      'worst radius', 'worst texture', 'worst perimeter', 'worst area',
      'worst smoothness', 'worst compactness', 'worst concavity',
      'worst concave points', 'worst symmetry', 'worst fractal dimension',
      'target'],
      dtype='object')
```

Missing Values:

```
mean radius      0
mean texture     0
mean perimeter   0
mean area        0
mean smoothness  0
mean compactness 0
mean concavity   0
mean concave points 0
mean symmetry    0
mean fractal dimension 0
radius error     0
texture error    0
perimeter error  0
area error       0
```

```

smoothness error      0
compactness error     0
concavity error       0
concave points error  0
symmetry error        0
fractal dimension error 0
worst radius          0
worst texture         0
worst perimeter       0
worst area            0
worst smoothness      0
worst compactness     0
worst concavity       0
worst concave points  0
worst symmetry        0
worst fractal dimension 0
target               0
dtype: int64

```

```

Target Names:
['malignant' 'benign']

```

```

# Display first 5 rows
df.head()

```

|          | mean<br>radius | mean<br>texture | mean<br>perimeter | mean<br>area | mean<br>smoothness | mean<br>compactness | mean<br>concavity | mean<br>concave points |
|----------|----------------|-----------------|-------------------|--------------|--------------------|---------------------|-------------------|------------------------|
| <b>0</b> | 17.99          | 10.38           | 122.80            | 1001.0       | 0.11840            | 0.27760             | 0.3001            | 0.147                  |
| <b>1</b> | 20.57          | 17.77           | 132.90            | 1326.0       | 0.08474            | 0.07864             | 0.0869            | 0.070                  |
| <b>2</b> | 19.69          | 21.25           | 130.00            | 1203.0       | 0.10960            | 0.15990             | 0.1974            | 0.127                  |
| <b>3</b> | 11.42          | 20.38           | 77.58             | 386.1        | 0.14250            | 0.28390             | 0.2414            | 0.105                  |
| <b>4</b> | 20.29          | 14.34           | 135.10            | 1297.0       | 0.10030            | 0.13280             | 0.1980            | 0.104                  |

```
5 rows × 9 columns
```

```

# Statistical Summary
df.describe()

```

|              | mean<br>radius | mean<br>texture | mean<br>perimeter | mean<br>area | mean<br>smoothness | mean<br>compactness |
|--------------|----------------|-----------------|-------------------|--------------|--------------------|---------------------|
| <b>count</b> | 569.000000     | 569.000000      | 569.000000        | 569.000000   | 569.000000         | 569.000000          |
| <b>mean</b>  | 14.127292      | 19.289649       | 91.969033         | 654.889104   | 0.096360           | 0.104341            |
| <b>std</b>   | 3.524049       | 4.301036        | 24.298981         | 351.914129   | 0.014064           | 0.052813            |
| <b>min</b>   | 6.981000       | 9.710000        | 43.790000         | 143.500000   | 0.052630           | 0.019380            |
| <b>25%</b>   | 11.700000      | 16.170000       | 75.170000         | 420.300000   | 0.086370           | 0.064920            |
| <b>50%</b>   | 13.370000      | 18.840000       | 86.240000         | 551.100000   | 0.095870           | 0.092630            |
| <b>75%</b>   | 15.780000      | 21.800000       | 104.100000        | 782.700000   | 0.105300           | 0.130400            |
| <b>max</b>   | 28.110000      | 39.280000       | 188.500000        | 2501.000000  | 0.163400           | 0.345400            |

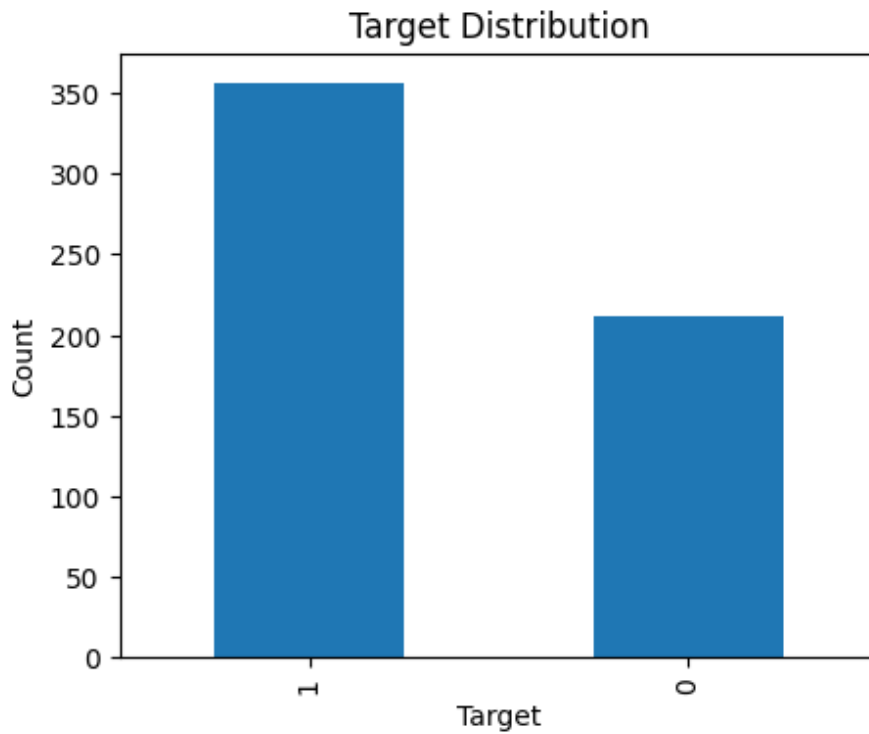
8 rows × 7 columns

```
# Check target distribution
print(df['target'].value_counts())
```

```
target
1    357
0    212
Name: count, dtype: int64
```

```
# Visualize target distribution
import matplotlib.pyplot as plt

plt.figure(figsize=(5,4))
df['target'].value_counts().plot(kind='bar')
plt.title("Target Distribution")
plt.xlabel("Target")
plt.ylabel("Count")
plt.show()
```



```
# Split the dataset into features and target

X = df.drop('target', axis=1)
y = df['target']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (455, 30)
Testing Data: (114, 30)
```

```
from sklearn.linear_model import LogisticRegression

# Create the model
model = LogisticRegression(max_iter=5000)

# Train the model
model.fit(X_train, y_train)

print("Model Trained Successfully!")
```

```
Model Trained Successfully!
```

```
# Make predictions

y_pred = model.predict(X_test)
```

```
print(y_pred[:10])
```

```
[1 0 0 1 1 0 0 0 1 1]
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 0.956140350877193
```

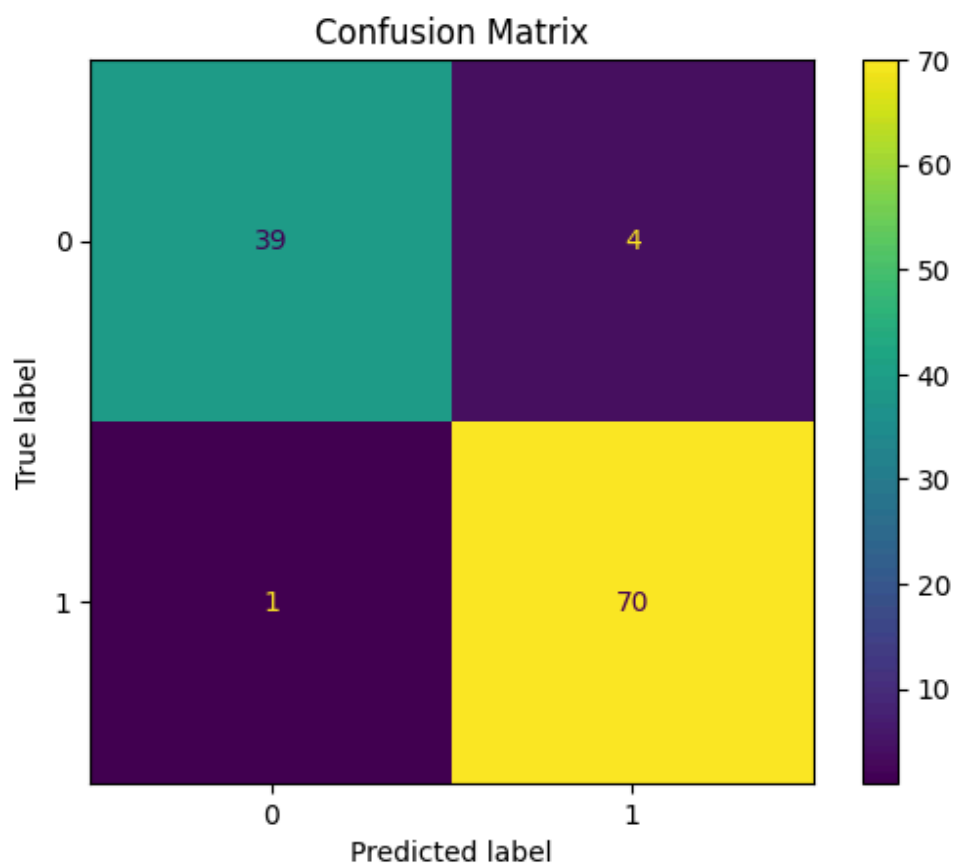
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm)  
disp.plot()
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```



```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, y_pred))
```

|   | precision | recall | f1-score | support |
|---|-----------|--------|----------|---------|
| 0 | 0.97      | 0.91   | 0.94     | 43      |

|              |      |      |      |     |
|--------------|------|------|------|-----|
| 1            | 0.95 | 0.99 | 0.97 | 71  |
| accuracy     |      |      | 0.96 | 114 |
| macro avg    | 0.96 | 0.95 | 0.95 | 114 |
| weighted avg | 0.96 | 0.96 | 0.96 | 114 |

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(random_state=42)

rf_model.fit(X_train, y_train)
```